MySQL Module-Assignment 3 – E-Learning Platform Purchase Analysis
Analyzing E-Learning Platform Purchases Using MySQL

Created By: D. Suganya

Course Name: AI-Driven Data Analytics

1. Project Overview and Objective

Project Overview

This assignment focuses on designing, querying, and analyzing an **E-Learning Platform Purchase Database** using MySQL. The database consists of three main tables—**Learners, Courses, and Purchases**—with relationships established through primary and foreign keys. The assignment demonstrates relational database design, data insertion, multi-table joins, aggregation, analytical queries, subqueries, CTEs, CASE expressions, NULL handling, and SQL views.

Objective

The primary objective is to analyze learner purchase data and generate meaningful business insights related to learner spending, course popularity, category performance, and purchasing behavior.

The assignment covers database creation, table relationships, sample data insertion, data exploration using joins, analytical SQL queries, advanced subqueries, CTEs, conditional classification, NULL handling, and view creation.

---

2. Problem Statement

- Create a database for the E-Learning Platform.
- Create **Learners, Courses, and Purchases** tables with appropriate data types.
- Establish primary key and foreign key relationships.
- Insert sample learner, course, and purchase records.
- Combine tables using INNER JOIN, LEFT JOIN, and RIGHT JOIN.
- Calculate learner-level spending and course-level purchase quantities.
- Analyze category-wise revenue and learner behavior.
- Identify learners spending above average.
- Identify courses that have never been purchased.
- Use CTE, CASE, IFNULL/COALESCE, subqueries, and views for analysis.
- Create a category performance view for reusable reporting.

---

3. Tools & Technologies

- **MySQL**
- **SQL**
- **MySQL Workbench / SQL Environment**

---

4. Database Structure

The E-Learning Platform database consists of three core tables. The **Purchases** table connects the **Learners** and **Courses** tables through learner_id and course_id foreign key relationships.

Table	Main Purpose	Key Attributes
Learners	Stores learner information	learner_id, full_name, country
Courses	Stores course information	course_id, course_name, category, unit_price
Purchases	Stores purchase transactions	purchase_id, learner_id, course_id, quantity, purchase_date

Database Relationship

Learners
  |
  | learner_id
  |
Purchases
  |
  | course_id
  |
Courses

---

5. Database Setup and Data Entry

The database was created using MySQL, followed by the creation of the three relational tables.

Primary keys were used to uniquely identify records, while foreign keys were used to establish relationships between learners, purchases, and courses.

Sample records were inserted into:

- Learners table
- Courses table
- Purchases table

The inserted records were verified using SELECT statements.

---

6. Data Exploration Using Joins

Joins were used to combine learner, course, and purchase information.

INNER JOIN

Used to display matching learner, purchase, and course records.

LEFT JOIN

Used to display all learners or courses, including records without matching purchases.

RIGHT JOIN

Used to demonstrate retrieval of records from the right-side table along with matching records from the left-side table.

The analysis includes:

- Learner Name
- Course Name
- Category
- Quantity
- Total Amount
- Purchase Date

Total purchase amount was calculated using:

Quantity × Unit Price

Currency values were formatted to two decimal places where required.

---

7. Analytical Queries

Q1 – Learner Total Spending

Calculated the total spending of each learner along with their country.

SQL concepts used:

- SUM()
- GROUP BY
- JOIN

```sql
/*Core Analytical Queries*/
/*Q1:Display each learner's total spending with their countr
SELECT l.full_name AS learner_name,
l.country AS country,
ROUND(SUM(c.unit_price * p.quantity),2) AS Total_spend
FROM learners l
JOIN purchases p
ON l.learner_id = p.learner_id
JOIN courses c
ON c.course_id = p.course_id
GROUP BY l.learner_id,
l.full_name,
l.country
ORDER BY Total_spend DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

learner_name	country	Total_spend
Divya	India	210000.00
ramya	USA	205000.00
Gopi	USA	120000.00
muthu	Indai	55000.00
Suganya	Malaysia	50000.00

Q2 – Top 3 Most Purchased Courses

Identified the top three courses based on purchase quantity.

SQL concepts used:

- SUM()
- GROUP BY
- ORDER BY
- LIMIT

```sql
SELECT c.course_name AS course_name,
sum(p.quantity) AS total_quantity
FROM courses c
JOIN purchases p
ON c.course_id = p.course_id
GROUP BY c.course_id,c.course_name
ORDER BY total_quantity DESC
LIMIT 3;
```

Result Grid | Filter Rows: | Export:

course_name	total_quantity
Data analyst	4
SAP	3
delevoping	3

Q3 – Category Performance

Calculated category-wise:

- Total revenue
- Number of unique learners

SQL concepts used:

- SUM()
- COUNT(DISTINCT)
- GROUP BY

```sql
SELECT c.category AS category,
ROUND(SUM(c.unit_price * p.quantity),2) AS Total_revenue,
COUNT(DISTINCT(p.learner_id)) AS Unique_learners
FROM courses c
JOIN purchases p
ON c.course_id = p.course_id
GROUP BY c.category
ORDER BY Unique_learners DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	Total_revenue	Unique_learners
advanced	240000.00	2
Beginner	145000.00	2
Intermediate	255000.00	2

Q4 – Learners Purchasing from Multiple Categories
Identified learners who purchased courses from more than one category.
SQL concepts used:

- JOIN
- COUNT(DISTINCT)
- GROUP BY
- HAVING

```
SELECT l.full_name AS learners_name,
COUNT(DISTINCT(c.category)) AS categary_count
FROM learners l
JOIN purchases p
ON l.learner_id = p.learner_id
JOIN courses c
ON p.course_id = c.course_id
GROUP BY l.learner_id,
l.full_name
HAVING categary_count > 1;
```

learners_name	categary_count
Divya	2

Q5 – Courses Never Purchased
Identified courses that do not have any associated purchase records.
SQL concepts used:

- LEFT JOIN
- NULL checking

---

8. Subqueries and Correlated Subqueries

Q6 – Learners Above Average Spending
Identified learners whose total spending is greater than the average learner spending.
SQL concepts used:

- Subquery
- AVG()
- SUM()
- HAVING

```
SELECT l.full_name AS learners,
SUM(c.unit_price * p.quantity) AS Total_spending
FROM learners l
LEFT JOIN purchases p
ON l.learner_id =p.learner_id
LEFT JOIN courses c
ON c.course_id = p.course_id
GROUP BY l.learner_id,l.full_name
HAVING Total_spending >
( SELECT AVG(Spending)
FROM
 (
SELECT  p.learner_id,
SUM(c.unit_price * p.quantity) AS Spending
FROM purchases p
left join courses c
on c.course_id = p.course_id
GROUP BY p.learner_id ) AS Learners_spending
);
```

learners	Total_spending
Divya	210000.00
ramya	205000.00

Q7 – Courses Above Beginner Category Prices

Identified courses whose price is higher than any course in the **Beginner** category.

SQL concepts used:

- Multi-row subquery
- ANY
- WHERE

Q8 – Learners Above Country Average

Identified learners whose spending is higher than the average spending of learners from their own country.

SQL concepts used:

- Correlated subquery
- AVG()
- SUM()
- GROUP BY
- HAVING

The country condition ensures that each learner is compared with the average spending of their respective country.

---

9. CTE, CASE and NULL Handling

Q9 – CTE: Learner Spending Above ₹10,000

A Common Table Expression was used to calculate total spending for each learner. The resulting temporary CTE was then filtered to display learners whose spending was above ₹10,000.

SQL concept used:

WITH

```
with learner_spending AS
(
SELECT l.learner_id,
l.full_name,
SUM(c.unit_price * p.quantity) AS total_spending
FROM learners l
LEFT JOIN purchases p
ON l.learner_id = p.learner_id
LEFT JOIN courses c
ON c.course_id = p.course_id
GROUP BY l.learner_id,l.full_name
)
SELECT *
FROM learner_spending
WHERE total_spending > 10000;
```

id | Filter Rows: | Export: | Wrap

er_id	full_name	total_spending
	Divya	210000.00
	ramya	205000.00
	Gopi	120000.00
	Suganya	50000.00
	muthu	55000.00

Q10 – Learner Spending Classification

A CASE expression was used to classify learners based on total spending.

Spending	Classification
Above ₹15,000	High Value
₹8,000 – ₹15,000	Medium Value
Below ₹8,000	Low Value

```sql
SELECT l.full_name as learners_name,
SUM(c.unit_price * p.quantity) AS total_spending,
CASE
WHEN SUM(c.unit_price * p.quantity)> 15000
THEN 'High Value'
WHEN SUM(c.unit_price * p.quantity) between 8000 and 15000
THEN 'Medium Value'
ELSE 'Low Value'
END AS Spending
FROM learners l
LEFT JOIN purchases p
ON l.learner_id = p.learner_id
LEFT JOIN courses c
ON c.course_id = p.course_id
GROUP BY l.learner_id,l.full_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

learners_name	total_spending	Spending
Divya	210000.00	High Value
ramya	205000.00	High Value
Gopi	120000.00	High Value
Suganya	50000.00	High Value
muthu	55000.00	High Value

Q11 – NULL Handling

IFNULL() / COALESCE() was used to replace NULL purchase quantities with zero for courses without purchase records.

Example:

COALESCE(SUM(p.quantity), 0)

This ensures that missing purchase values are represented as 0.

```sql
Display all courses and replace NULL purchase cou
COALESCE()*/
SELECT
c.course_id,
c.course_name,
COALESCE(SUM(p.quantity), 0) AS purchase_count
FROM courses c
LEFT JOIN purchases p
ON c.course_id = p.course_id
GROUP BY
c.course_id,
c.course_name;
```

Result Grid | Filter Rows: | Export:

course_id	course_name	purchase_count
1	Data analyst	2
2	SAP	3
3	Data analyst	4
4	delevoping	3
5	fullstack	1

Q12 - View Creation

A SQL view named:

category_performance_view

was created to provide reusable category-level performance information.

The view contains:

- Category
- Total Revenue
- Number of Purchases
- Average Revenue per Purchase

The view can be accessed using:

```
SELECT *
FROM category_performance_view;
```

```sql
CREATE VIEW category_performance_view AS
SELECT
c.category,
SUM(c.unit_price * p.quantity) AS total_revenue,
COUNT(p.purchase_id) AS number_of_purchases,
SUM(c.unit_price * p.quantity) / COUNT(p.purchase_id)
AS average_revenue_per_purchase
FROM courses c
LEFT JOIN purchases p
ON c.course_id = p.course_id
GROUP BY c.category;

SELECT *
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	total_revenue	number_of_purchases	average_revenue_per_purchase
Beginner	145000.00	2	72500.000000
Intermediate	255000.00	3	85000.000000
advanced	240000.00	2	120000.000000

---

10. Skills Demonstrated

- MySQL database and schema creation.
- Relational database design using Primary Key and Foreign Key relationships.
- Data insertion and verification.
- Multi-table querying using INNER, LEFT, and RIGHT JOIN.
- Aggregation using SUM(), COUNT(), and AVG().
- Filtering using WHERE and HAVING.
- Sorting and limiting results using ORDER BY and LIMIT.
- Analytical calculations using SQL expressions.
- Single-row and multi-row subqueries.
- Correlated subqueries.
- CTE implementation using WITH.
- Conditional classification using CASE.
- NULL handling using IFNULL() and COALESCE().
- SQL View creation for reusable reporting.

---

11. Key Outcomes

- Successfully designed a relational E-Learning Purchase Database.
- Established relationships between Learners, Courses, and Purchases.
- Analyzed learner-level spending and purchasing behavior.
- Identified popular courses and high-performing categories.
- Identified learners purchasing across multiple categories.
- Identified courses with no purchase records.
- Applied subqueries and correlated subqueries for advanced analysis.
- Used CTE and CASE expressions for structured analysis and learner classification.
- Applied NULL handling to improve result presentation.
- Created a category performance view for reusable analysis.

The sample documentation also presents outcomes as concise achievement statements before the conclusion, so this section matches that style.

---

12. Conclusion

This MySQL assignment demonstrates the practical implementation of relational database design and SQL-based data analysis for an E-Learning Platform. The project covers database

and table creation, primary and foreign key relationships, data insertion, joins, aggregation, analytical queries, subqueries, CTEs, CASE expressions, NULL handling, and SQL views. The analysis provides a structured approach to understanding learner spending, course popularity, category performance, and purchasing behavior. The SQL queries and execution screenshots document the implementation and verification of the analysis.
